Template

Table of Contents

1. Introduction and Goals . . . 3
   1.1. Requirements Overview . . . 3
   1.2. Quality Goals . . . 3
   1.3. Stakeholders . . . 4
2. Architecture Constraints . . . 5
   2.1. Technical Constraints . . . 5
   2.2. Organizational Constraints . . . 5
   2.3. Conventions . . . 5
3. Context and Scope . . . 6
   3.1. Business Context . . . 6
   3.2. Technical Context . . . 6
4. Solution Strategy . . . 8
5. Building Block View . . . 10
   5.1. Whitebox Overall System . . . 10
6. Runtime View . . . 21
   6.1. User Authentication . . . 21
   6.2. Create/Update Workout . . . 21
   6.3. Delete Workout . . . 22
7. Deployment View . . . 23
   7.1. Motivation . . . 23
   7.2. Quality and/or Performance Features . . . 24
8. Cross-cutting Concepts . . . 25
   8.1. Domain Models . . . 25
   8.2. Persistence . . . 29
   8.3. Security & Session Handling . . . 29
   8.4. User Interface . . . 29
   8.5. Communication and Messaging . . . 29
   8.6. Resilience and Fault Tolerance . . . 30
   8.7. Error Handling and Validity checks . . . 30
   8.8. Observability (Loggin and Tracing) . . . 30
   8.9. Configurability and Build Management . . . 30
9. Architecture Decisions . . . 31
   9.1. HTTP 202 Accepted . . . 31
   9.2. ADR: SNS + SQS Fanout & Service Decoupling . . . 31

9.3. SSE vs websockets . . . 32
9.4. Hybrid JWT-Redis Session Management . . . 32
9.5. Gemini API . . . 32
10. Quality Requirements . . . 33
10.1. Quality Requirements Overview . . . 33
10.2. Quality Scenarios . . . 33
11. Risks and Technical Debts . . . 35
12. Glossary . . . 36

1. Introduction and Goals

Fitness Tracker is a project to:

- Track my strength workouts and analyze overall progress with AI-generated insights
- Experiment with microservices architecture
- Use industry standard software architecture documentation
- Sharpen Java, Python, and SQL development skills

1.1. Requirements Overview

Contents

The purpose of Fitness Tracker is to provide a way for users to keep track of their strength training workouts and receive AI-generated recommendations and analysis about their progress.

This project is also intended to explore a microservices architecture and utilize different programming languages and technologies.

Main Features

- Users can create, update, and delete workout plans
- Users can view their workout history
- AI-generated analysis of completed workouts
- Simple UI for interacting with the system
- Cloud-native microservices architecture

The application should be able to handle multiple users and admins. Users should only be able to see their own workouts and recommendation data. Admins can only perform defined admin functionalities and cannot view user data.

The application should be able to store an "unlimited" number of user workouts.

The application will use a starter list of exercises from https://strengthtoovercome.com/functional-fitness-exercise-database

1.2. Quality Goals

Table 1. Quality Goals

No.	Quality	Motivation
1	Understandability	Each microservice serves a clear, isolated purpose. This modularity ensures that the system remains easy to navigate and modify, especially when adding new functional features. Documentation is well maintained.

No.	Quality	Motivation
2	Usability	Interface should be intuitive and easy to enter workouts.
3	Testability	All microservices should be well tested and documented. A 70% code coverage threshold ensures that core domain logic and event-processing paths are validated.

1.3. Stakeholders

Table 2. Stakeholders

Name	Goal
Developers	Backend developers who are interested in event driven architecture, AI integration, Java and Python best practices.
Fitness Enthusiasts	Everyone from beginner gym-goers looking for a solution to maintain consistent progress to advanced athletes looking to level up their programs.
Software Architects	A project to experiment with microservices architecture and documentation best practices.

2. Architecture Constraints

2.1. Technical Constraints

Table 3. Technical Constraints

Constraint	Goal
Java 25 / Maven	Chosen as primary backend language to ensure long term support and high performance. Spring Boot has a mature ecosytem with integrations for spring security and spring cloud, which are critical features of this application.
Python 3.12	Required for the Analytics Service to provide compatibility with ML libraries and AI processing frameworks.
LocalStack	Provides AWS SNS/SQS locally without incurring cloud costs.
Docker / Kubernetes	Application must be fully functional in local k8s environment. This ensures easy future migration to the cloud.

2.2. Organizational Constraints

Table 4. Organizational Constraints

Constraint	Goal
Solo Developer Project	Architecutre must be manageable by one person and require minimal operational overhead.
4 month timeline	Primarily an exploration/MVP project, with options to design for scalability later on.
70% code coverage goal	Testing is paramount. Metric for learning and quality.
Licensing	Publish under liberal open source license to encourage learning.

2.3. Conventions

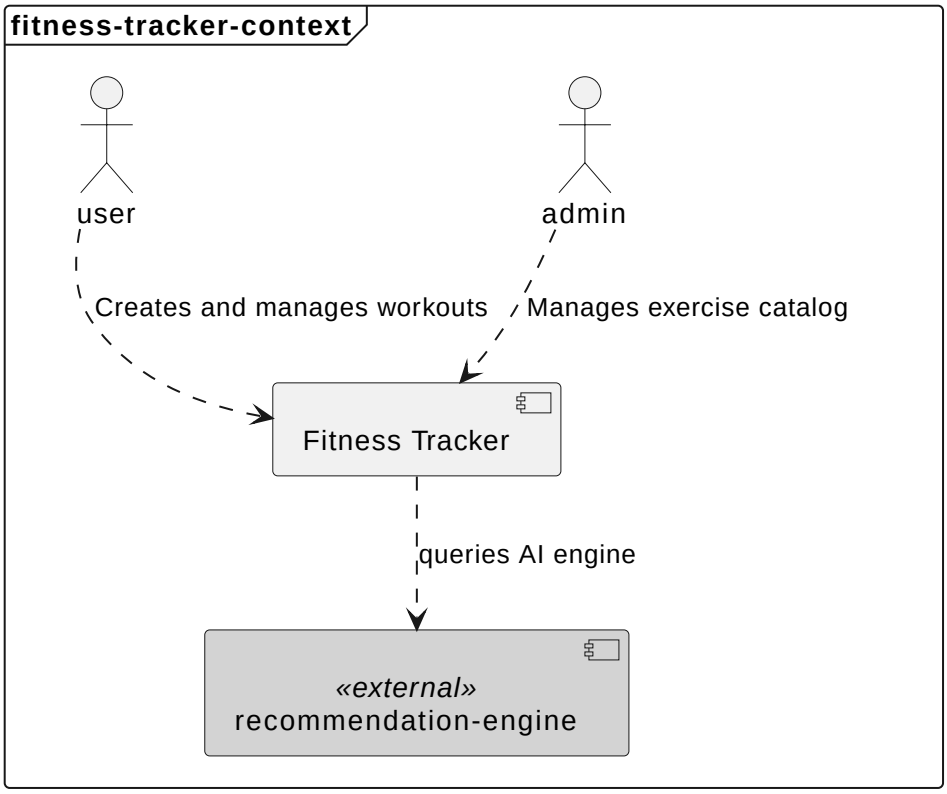
Table 5. Conventions

Convention	Goal
Architecture Documentation	arc42 template
Coding conventions	Java Conventions and PEP 8

3. Context and Scope

This section describes who uses Fitness Tracker and what external systems it interacts with.

3.1. Business Context



User

A fitness enthusiast uses Fitness Tracker to log their strength training workouts and receive AI-generated workout plans.

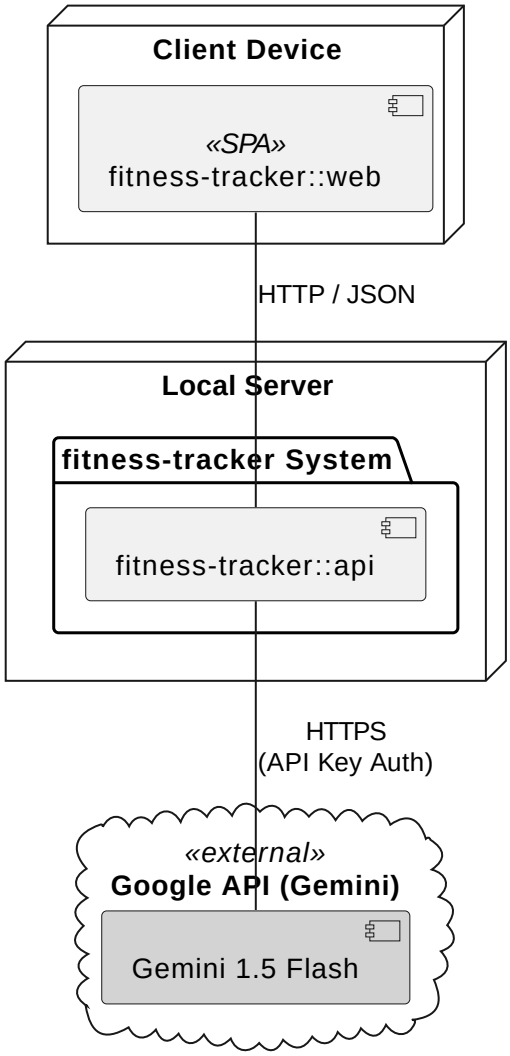
Admin

An admin manages the list of exercises available to users.

AI Engine

The free Gemini API will be used with developer-authored prompts for each type of supported analytical query (ie to generate a summary of the workout or to provide recommendations for the user's next workout).

3.2. Technical Context



Backend (fitness-tracker::api)

The backend runs as microservices within a local kubernetes cluster. REST endpoints are exposed over HTTP for the frontend. The backend will make HTTPS requests to the Gemini API using an API Key.

Frontend (fitness-tracker::web)

The React frontend runs in a web browser. It communicates with the backend over HTTP. It will service both Admins and Users.

Google Gemini API

The backend makes HTTP requests to Gemini's REST API supplying a prompt along with the workout payload to generate workout analysis and recommendations.

4. Solution Strategy

The goal of Fitness Tracker is to design and implement a cloud-native microservices ecosystem and explore design decisions related to scalability, fault tolerance and resiliency.

Runtime Stack

A benefit of microservices architecture is the flexibility to select an optimal runtime stack for each service.

- **Primary Backend (Java 25 / Spring Boot):** The majority of services will be written with Spring Boot and Java 25. Spring Boot is frequently used in high quality production environments and provides support for many features needed in this project (spring security, spring cloud gateway, spring data).
- **Experimental Evaluation (Golang):** While Java is the primary backend langauge, Golang will be used for future service implementations to provide a comparison against the JVM stack. This choice serves as a technical refresher for two key technologies utilized throughout my career.
- **AI (Python 3.12):** The Analytics Service is written in Python, given that it's the leading stack for AI and machine learning. Although the service currently functions as an orchestrator for Gemini REST API calls, this choice ensures the architecture is future-proofed for integration of custom-tailored models or LLM frameworks.

Database Selection

Multiple databases are used within this project to optimize for data integrity, flexible schemas and performance. The system follows a database-per-service pattern to ensure service autonomy and fault tolerance.

- **Workout and User Management (Relational/PostgreSQL):** The Workout Management Service and User Service maintain independent PostgreSQL instances. Relationships across this service boundary are maintained via UUIDs. The fitness database uses a normalized structure with three tables to capture complex workout data. PostgreSQL ensures ACID compliance, guaranteeing that fitness transactions are processed reliably and data remains consistent and easily queryable.
- **Notification Management (Document/MongoDB):** The Notification database houses AI-generated messages. These messages are semi-structured and the structure could change as the AI prompts and models are refined. MongoDB allows for the storage of diverse message types and its native support for JSON-like documents aligns with the Gemini REST API.
- **Session Management (In-Memory/Redis):** Redis will serve as a high speed registry for valid JWT identifiers. This allows for instant session revocation. By using Redis, we can have stateless microservices and achieve sub-millisecond latency for authentication checks. Additionally, the Redis Time-to-Live functionality automatically handles session expiration.

Security

This project will implement a hybrid session management solution leveraging the strengths of JWT and Redis. The API Gateway will require user authentication and authorization.

- **Authorization Service (Spring Security):** The Authorization Service is a central security

authority. It utilizes Spring Security to authenticate user credentials against the PostgresQL User DB and issues a cryptographically signed JWT. It registers the token's unique ID in Redis to activate the session. This token contains user claims, allowing downstream services to identify the user.

- **Hybrid Session Management (Redis Integration):** A limitation of JWTs is the inability to revoke them before they expire. To solve this problem, the architecture incorporates Redis as a session validation layer. This creates a hybrid model that combines the scalability of JWT with control of traditional sessions.
- **Identity Propagation & Internal Trust:** Identity Propagation will be used to propagate the authenticated user's identity to internal services. This allows the internal microservices to be secure while eliminating the need to perform redundant JWT validation or Redis lookups.

Frontend Backend Communication

The interaction between the client and the backend is designed to be stateless and secure. The system uses a central entrypoint through the API Gateway (Spring Cloud Gateway), which handles routing, rate limiting, and authentication.

Communication is handled by both synchronous and asynchronous REST (JSON/HTTP) interfaces. Standard operations are handled synchronously. Requests that trigger AI analysis use an asynchronous flow and return an HTTP 202 Accepted status to the client. The client maintains a SSE (Server-Sent Events) connection, allowing the backend to notify the client when the async workflow is complete.

Event Driven Architecture

The most interesting design component of this project is the event driven architecture. By implementing an HTTP 202 Accepted flow, the Workout Management Service can acknowledge client requests immediately while handling the processing of the request asynchronously. The design uses an SNS/SQS fan-out pattern, where the Workout Management Service publishes events to a central SNS topic that are then distributed to specialized SQS queues depending on event filter. This provides the flexibility to add more event types and processing modules in the future.

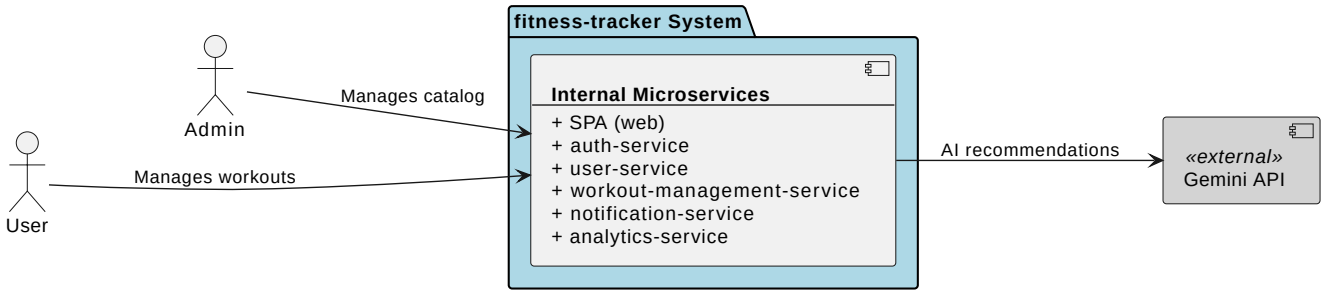
In addition this project uses SQS to decouple microservices to improve the horizontal scalability and fault tolerance of the solution. SQS acts as a buffer for long running Gemini API queries, ensuring the client isn't blocked by AI processing latency. Specifically, the Analytics Service is isolated between two SQS boundaries. This design decouples the high latency AI processing from both the input source and output destination, allowing the service to easily scale horizontally. While scaling is theoretical for this POC, the architecture is designed to solve common bottlenecks in production systems.

Deployment strategy

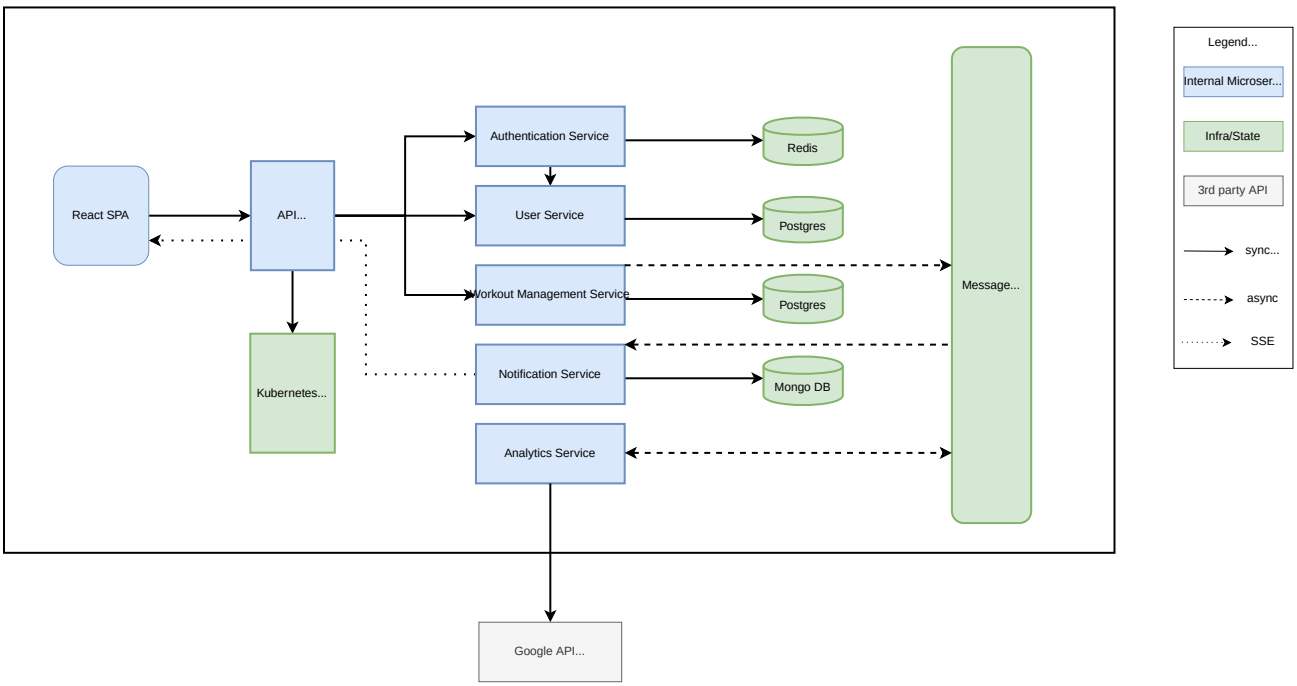
On the deployment front, Kubernetes was chosen because it is the leading container orchestration service. Kubernetes also has built in service discovery, which is helpful in a microservices environment.

5. Building Block View

Fitness Tracker is a microservices based application with a React SPA frontend and 5 microservices that make up the backend.



5.1. Whitebox Overall System



Motivation

This system is designed with a decoupled microservices architecture to ensure fault tolerance and independent scaling of core services. A primary factor for this design is the integration with the Google Gemini API, which introduces variable, high-latency responses and strict rate limiting. To prevent thread starvation and API Gateway timeouts, the system uses an asynchronous, event driven communication strategy for this workflow.

Contained Building Blocks

Name	Responsibility
React SPA	React Frontend for users and admins to interact with the system
API Gateway	Unified entrypoint for all client requests. Handles routing, rate limiting and session validation.

Name	Responsibility
Auth Service	Validates user credentials and issues JWTs. Uses Redis to store and revoke session liveliness tokens.
User Service	Manages user account lifecycle, profiles and roles. Persists user data to relational db.
Workout Management Service	Handles core business logic for tracking workouts. Publishes lifecycle events to Message Broker for downstream processing. Persists workout data to relational db.
Notification Service	Manages persistence of AI insights in MongoDB and pushes real time updates to UI via Server-Sent Events (SSE).
Analytics Service	Consumes workout events and orchestrates high latency calls to Google Gemini API for personalized analysis.
Message Broker	AWS SNS and SQS infrastructure that enables decoupled, event driven communication.

All of the above blackboxes are standalone functional microservices except for the Message Broker and the API Gateway, which are shared infrastructure components. The API Gateway acts as the centralized entrypoint for all client traffic and the Message Broker facilitates the system's asynchronous communication.

Important Interfaces

Name	Responsibility
Public API Gateway	The primary REST entry point for React SPA. Handles routing to User, Workout, and Notification services.
Auth/Identity API	Internal REST interface used by API Gateway to validate user credentials and sessions.
Workout Ingestion API (200/201/204)	Public REST interface for creating, updating and deleting workouts.
Workout Ingestion API (202 Accepted)	Public REST interface for completing workouts.
Workout Stream (async)	Message-based interface for publishing workout lifecycle events to subscribers.
Analytics Stream	Consumes events from Workout Stream via SQS to trigger AI anaylsis.
Notification Stream	Internal event based interface to trigger real time push notifications to users.
Gemini AI Integration	Outbound interface to Google Gemini API.
Persistence to DBs	JDBC interface between microservices and their respective databases.

5.1.1. React SPA (Blackbox)

Responsibility

The React SPA provides the user interface for both users and admins to interact with the Fitness Tracker system. The application is built as a standard Single Page Application with React.

Table 6. Interfaces

Name	Description
REST interface	Communicates with the API Gateway
SSE interface	Establishes long-lived, unidirectional connection with Notification Service to receive real-time AI workout analysis updates.

5.1.2. API Gateway (Blackbox)

Responsibility

The API Gateway serves as the single entry point for all client requests. It handles request routing, rate limiting, and security enforcement. It utilizes Kubernetes Service Discovery to resolve microserivce instances.

Table 7. Interfaces

Name	Description
REST Interface	Exposes public endpoint for clients (React SPA) to interact with.
Internal routing	Routes requests to appropriate microservices using Kubernetes Service Discovery.

5.1.3. Auth Service (Blackbox)

Responsibility

The Auth Service manages the security lifecycle of all user sessions. It performs credential verification, issues signed JWTs and manages session state.

Table 8. Interfaces

Name	Description
REST Interface `/login /logout`	Public REST API for credential exchange for token on login and session invalidation on logout.
Internal Redis	Store and manage jti claim and session metadata

5.1.4. User Service (Blackbox)

Responsibility

User service manages the lifecycle of a user (registration, profile, deactivation). It serves as source of truth for user identity. It provides necessary data for Auth Service to perform authentication and authorization checks.

Table 9. Interfaces

Name	Description
REST Interface `/api/v1/users`	Public REST API exposed via the API gateway for user registration and profile updates.
Internal REST Interface `/internal/v1/identity`	Internal REST API used by Auth Service to retrieve hashed credentials and roles.
PostgreSQL Persistence	Private JDBC interface to User DB for persisting users.

5.1.5. Workout Management Service (Blackbox)

Responsibility

Workout Management Service is the core service for managing the lifecycle of user workouts, supporting full CRUD operations. It acts as a producer for the asynchronous data stream, publishing completed workout data to be processed downstream by analytics services.

Table 10. Interfaces

Name	Description
Public REST Interface `/api/v1/workouts`	Public REST API for users to submit workout data and view workout history.
Workout Event Stream `Workout.Domain.Events`	Publisher to SNS Topic.
PostgreSQL Persistence	Private JDBC interface to Fitness db for persisting workout data.

5.1.6. Notification Service (Blackbox)

Responsibility

The Notification Service stores all notifications for the Client and alerts the Client upon new messages ready for display. It persists AI-generated analysis results and manages active Server-Sent Event (SSE) streams.

Table 11. Interfaces

Name	Description
SSE Stream `/api/v1/notifications/stream`	A unidirectional, long-lived HTTP connection used to push real-time alerts directly to React SPA.
Public REST interface `/api/v1/notifications`	Public REST API for retrieving and managing notifications.
MongoDB Persistence	Private MongoDB layer to persist notification data.
SQS Consumer	Internal listener that pulls processed analysis results from Notification Queue and cleanup/delete events from Cleanup Queue.

5.1.7. Analytics Service (Blackbox)

Responsibility

The Analytics service is the AI Orchestrator. It's a specialized consumer designed to handle the high-latency and unpredictable nature of LLM interactions. It transforms raw workout data into structured prompts for the Google Gemini API and publishes the resulting insights to the notification stream via SQS.

Table 12. Interfaces

Name	Description
Workout Event Stream `Workout.Domain.Events`	SQS Consumer that triggers AI analysis workflow
AI Analysis Client	Outbound HTTPS requests to Google API (Gemini). Handles api authentication, prompt injection and response parsing.
Notification Stream	An asynchronous SQS interface used to publish final AI generated insights to the Notification Service.

5.1.8. Message Broker (Blackbox)

Responsibility

The Message Broker provides the asynchronous communication backbone for the system. It's resonsible for reliable distribution of events to downstream services. By implementing a fan-out pattern with message filtering, it ensures that workout processing, data cleanup, and notification delivery are decoupled and handled independently.

Table 13. Interfaces

Name	Description
`Workout.Domain.Events`	SNS Topic that fans out to SQS
Analytics Queue `Filter: type=process`	SQS queue that receives workout completed events intended for AI processing.
Cleanup Queue `Filter: type=delete`	SQS queue that receives workout lifecycle events to ensure data is purged properly.
Notification Queue	SQS queue serving as a point to point buffer between Analytics and Notification services.

5.1.9. White Box: Notification Service

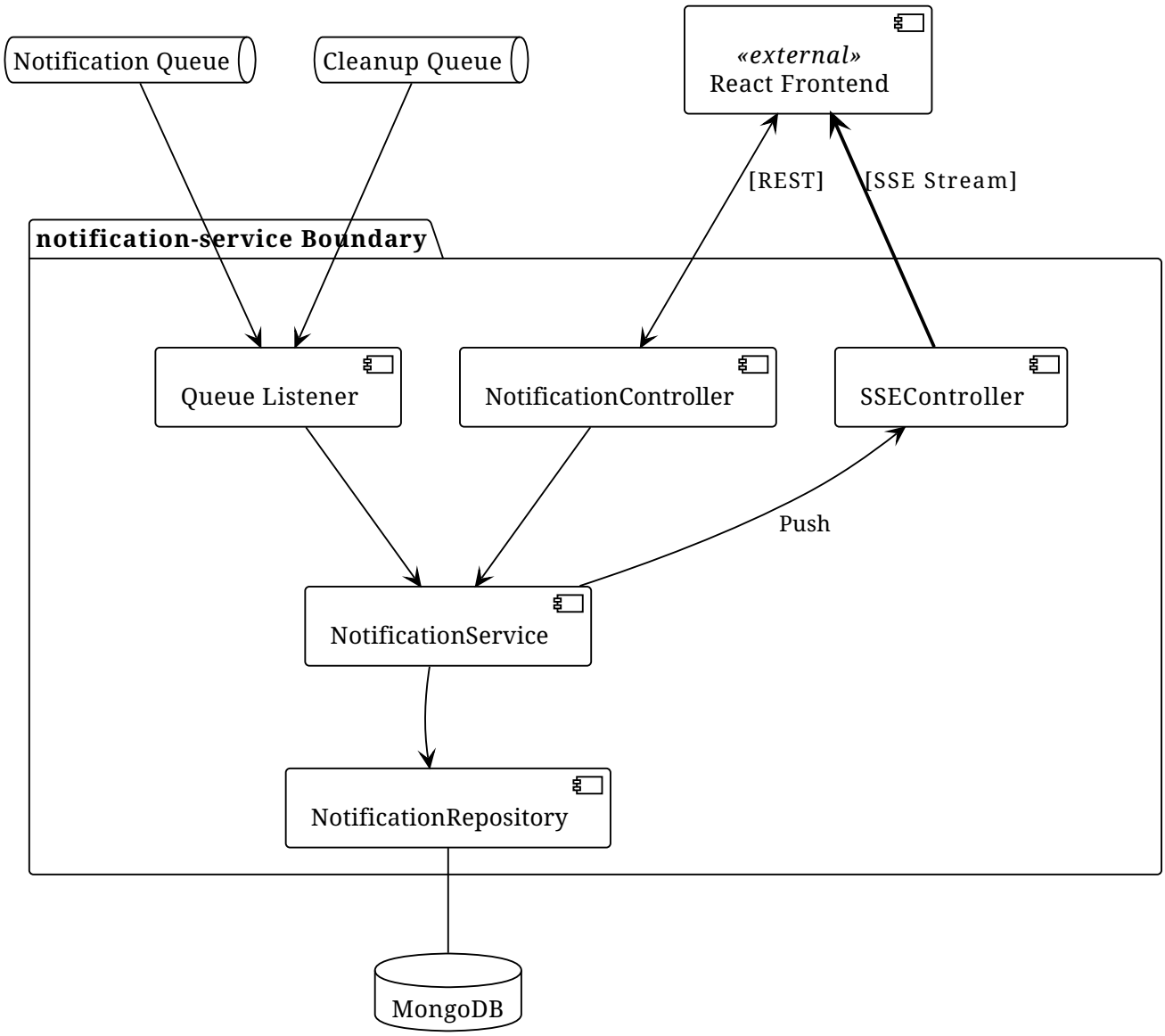


Table 14. Internal Components

Component	Responsibility
Queue Listener	Async entrypoint. Monitors the Notification Queue for incoming AI insights and the Cleanup Queue for deletion events.
NotificationController	REST controller that handles synchronous request-response interactions with React Frontend.
SSE Controller	Manages persistent Server-Sent Events pipe. Pushes real-time UI updates.
NotificationService	Core logic of service.
NotificationRepository	Data access layer. Encapsulates all interactions with MongoDB.

Table 15. Contained Blackboxes

Component	Responsibility
MongoDB	Document store that persists AI analysis message, read/unread states, and metadata linking notifications to specific users and workouts.
Cleanup Queue (SQS)	Buffer for delete events.
Notification Queue (SQS)	Point to point buffer between Analytics Service and Notification Service.

5.1.10. White Box: Workout Management Service

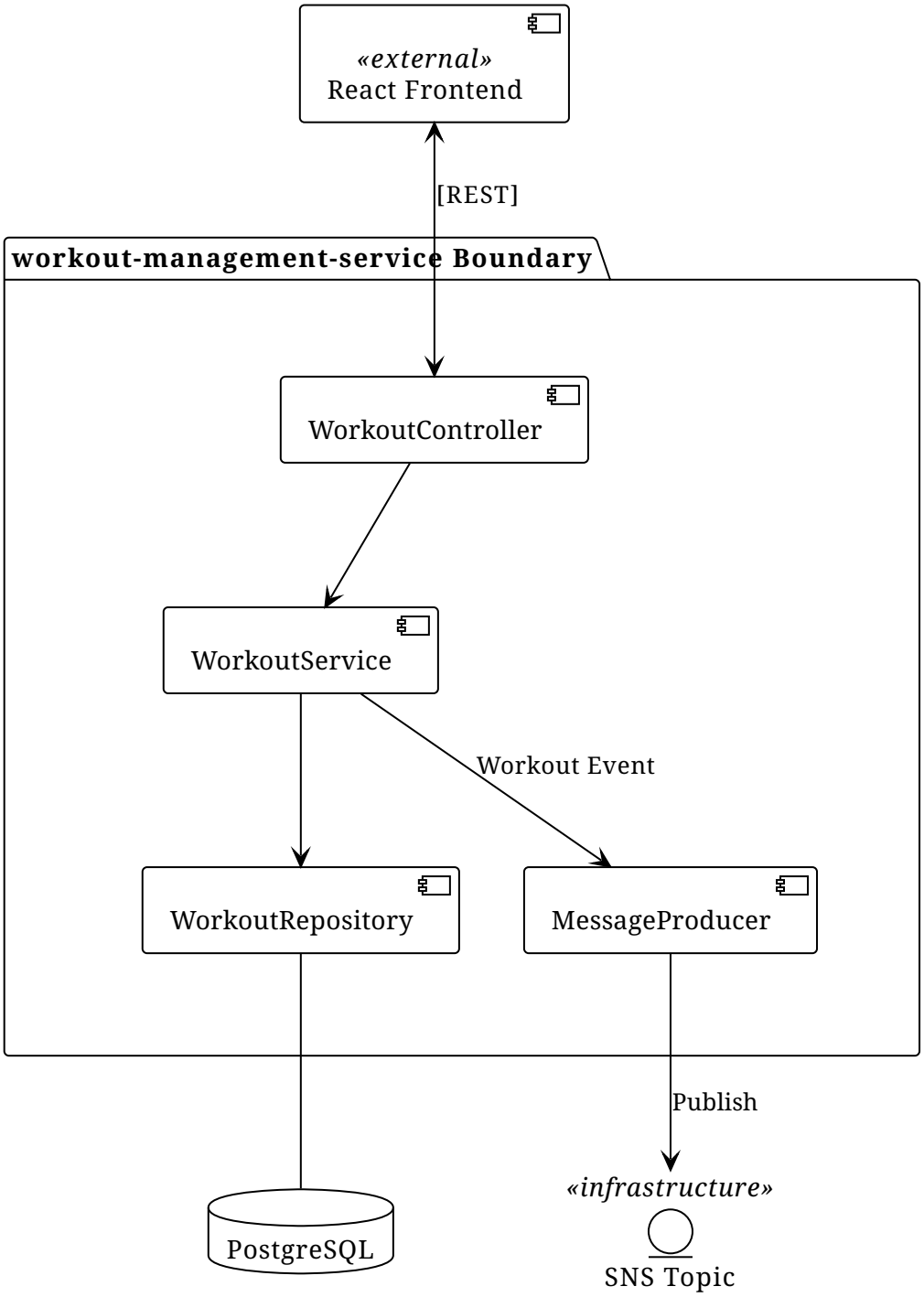


Table 16. Internal Components

Component	Responsibility
WorkoutController	REST interface that handles synchronous HTTP requests for CRUD operations on workouts. Returns 201 Created or 202 Accepted.
WorkoutService	Core business logic. Orchestrates workout lifecycle, ensuring data integrity before persistence and determining when to trigger downstream events.
WorkoutRepository	Data access layer for PostgreSQL. Manages persistence of workout entities, exercise mappings, and user specific workout details in relational store.
MessageProducer	Integration to Message Broker. Publishes events to SNS Topic. `Workout Completed` sent to Analytics Queue. `Workout Deleted` sent to Cleanup Queue

Table 17. Contained Blackboxes

Component	Responsibility
PostgreSQL	Relational storage for workout data.
SNS Topic	Entrypoint for async communication to downstream services.

5.1.11. White Box: Analytics Service

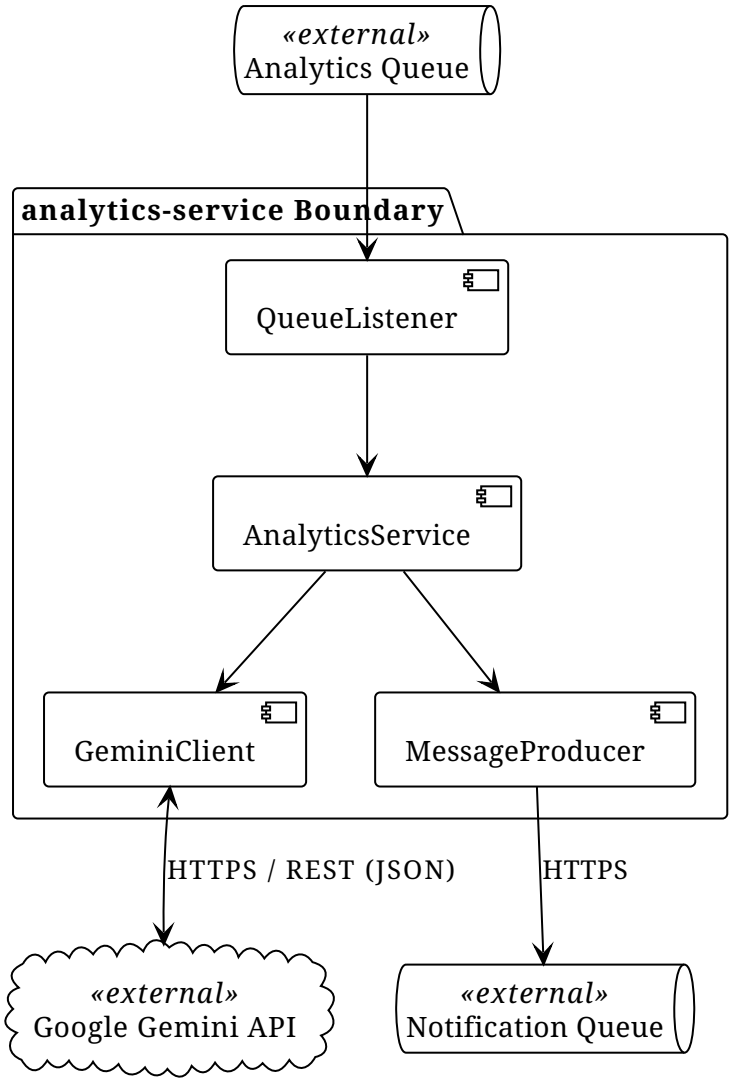


Table 18. Internal Components

Component	Responsibility
QueueListener	Async entrypoint. Monitors the Analytics Queue.
AnalyticsService	Core business logic. Creates AI prompt that includes workout data.
GeminiClient	AI integration adapter. Handles REST communication with Google Gemini API.
MessageProducer	Integration to Message Broker. Publishes AI generated insight messages to Notification Queue.

Table 19. Contained Blackboxes

Component	Responsibility
Google Gemini API	External AI provider. Processes workout data given a prompt. Treated as a blackbox as only interact via public API.
Analytics Queue (SQS)	Inbound buffer for messages that need to be processed.
Notification Queue (SQS)	Outbound buffer to send processed messages to Notification Service

5.1.12. White Box: Message Broker

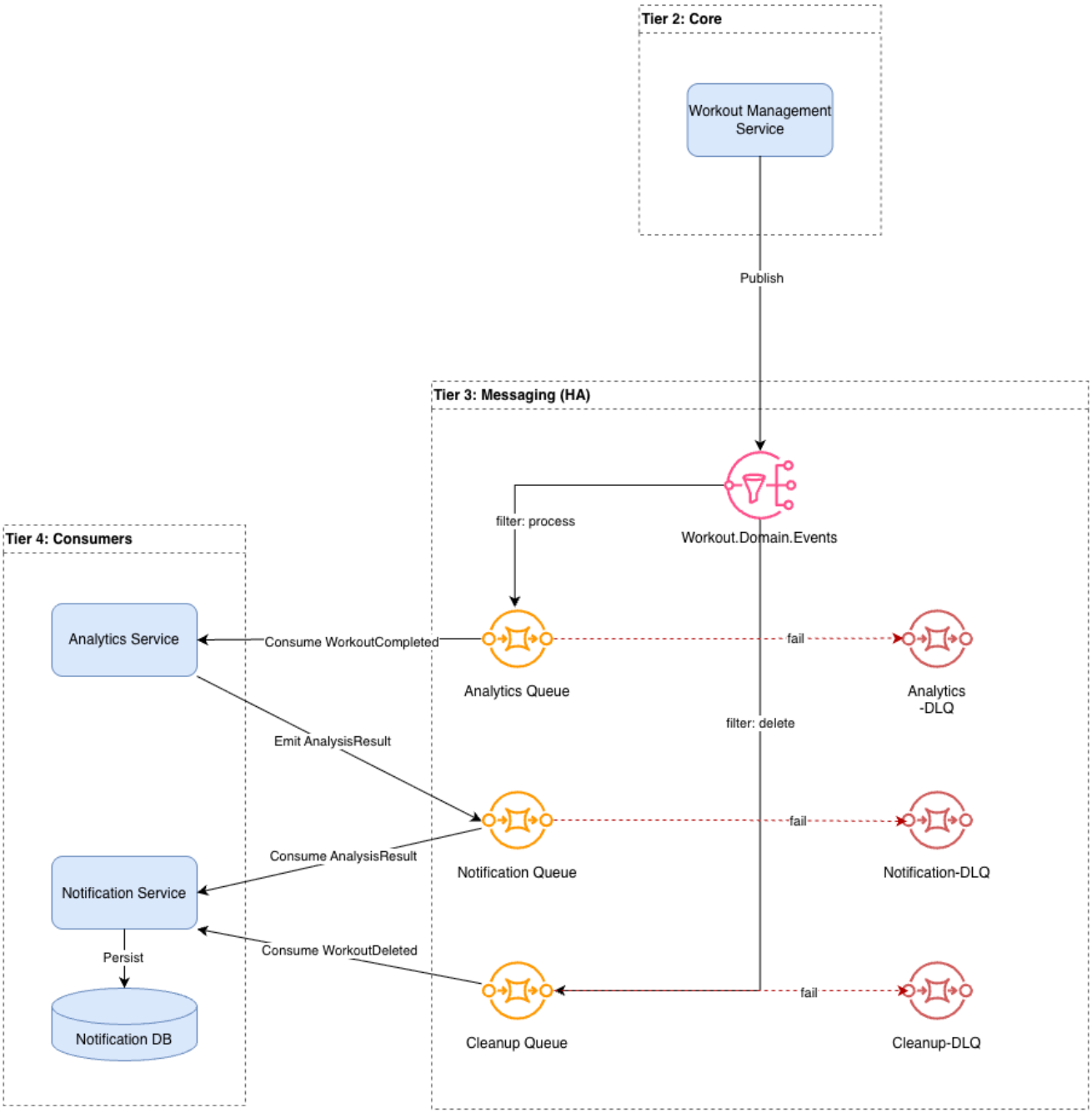


Table 20. Internal Components

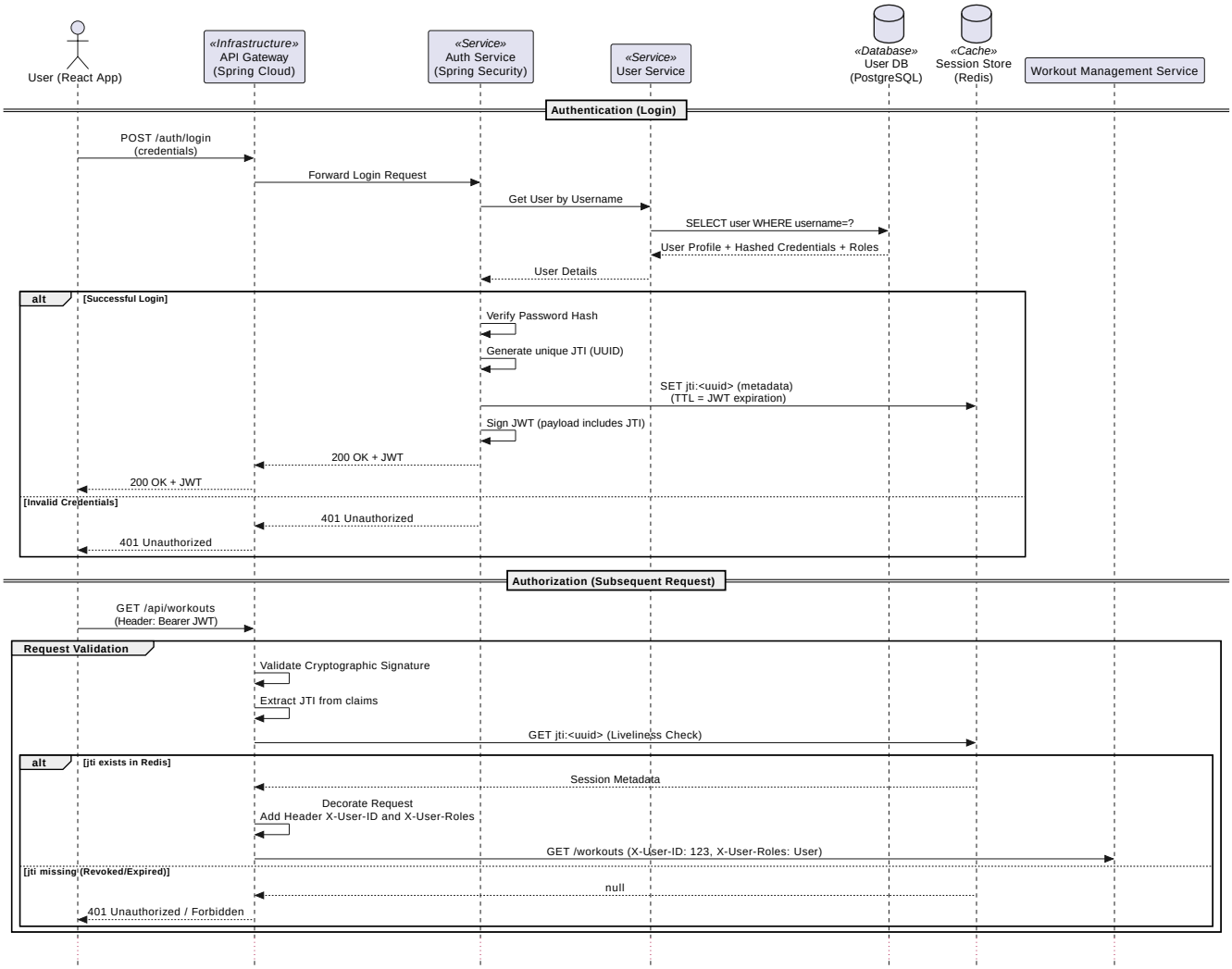
Component	Responsibility
Workout.Domain.Events (SNS)	Central entrypoint for event driven architecture. Evaluates message attributes to decide which downstream queue receives the message.
Analytics Queue (SQS)	Buffers events tagged with `process` filter for Analytics Service.
Cleanup Queue (SQS)	Buffers events tagged with `delete` filter to trigger data removal in Notification service.
Notification Queue (SQS)	Point to point buffer between Analytics Service and Notification Service.

Component	Responsibility
Dead Letter Queues	Store messages that have failed processing after multiple retries, allowing for manual recovery without blocking main queues.

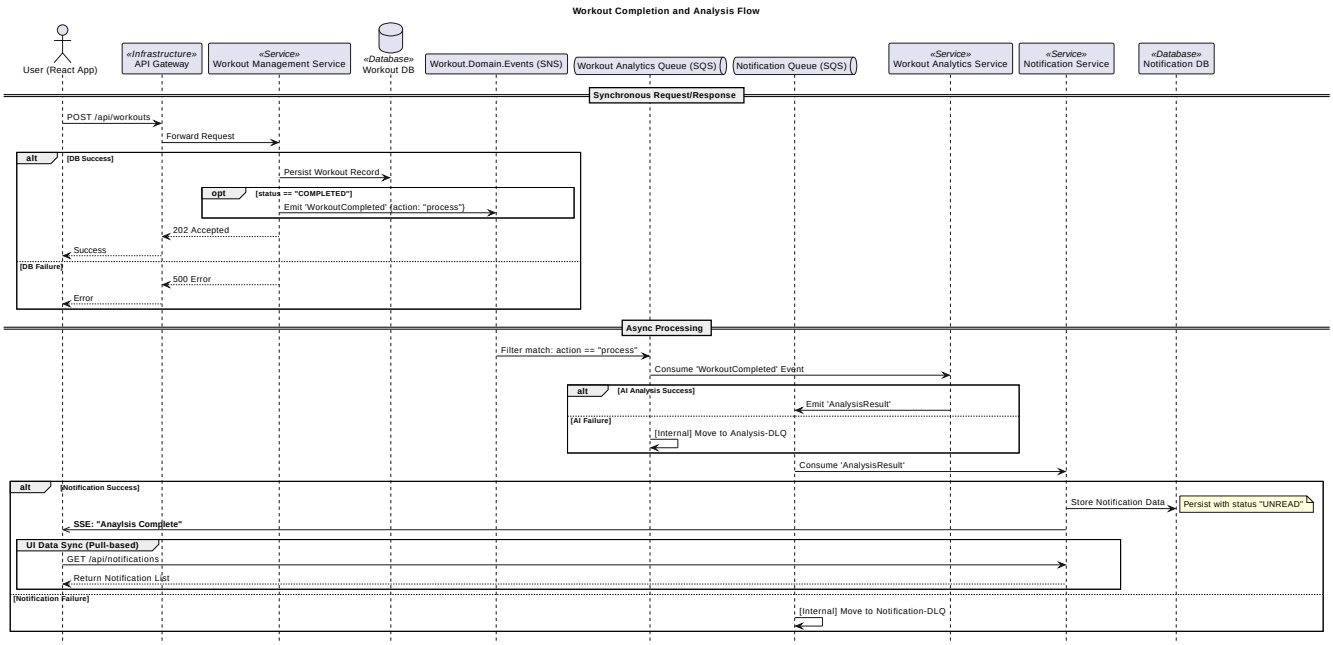
6. Runtime View

6.1. User Authentication

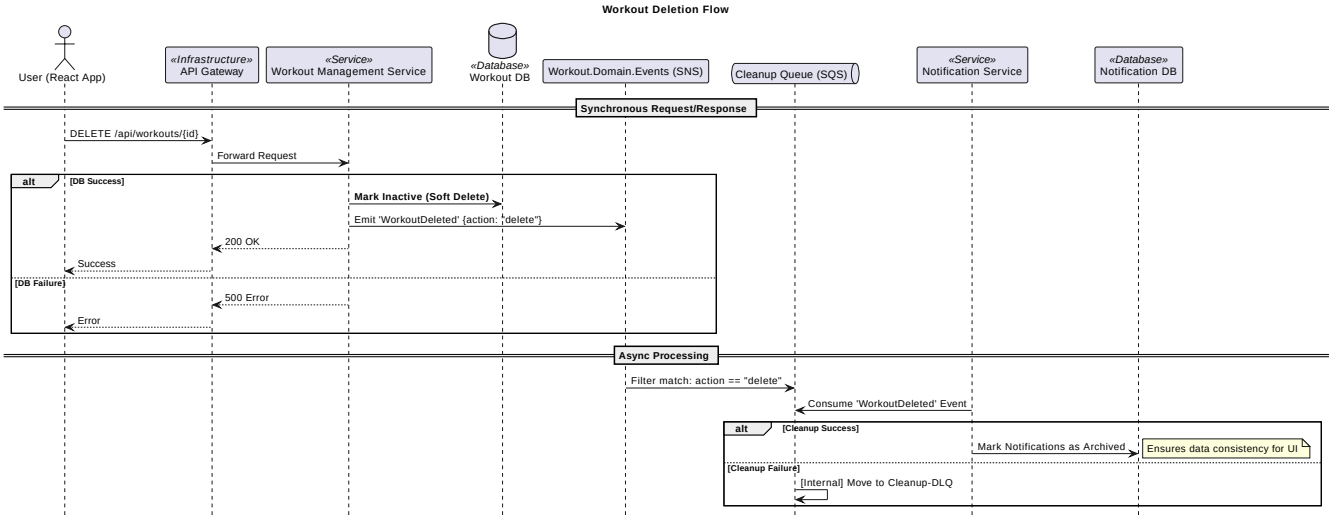
User Authentication is an important flow to highlight in this microservices architecture. Identity verification is centralized in the API Gateway before routing requests to the appropriate microservice.



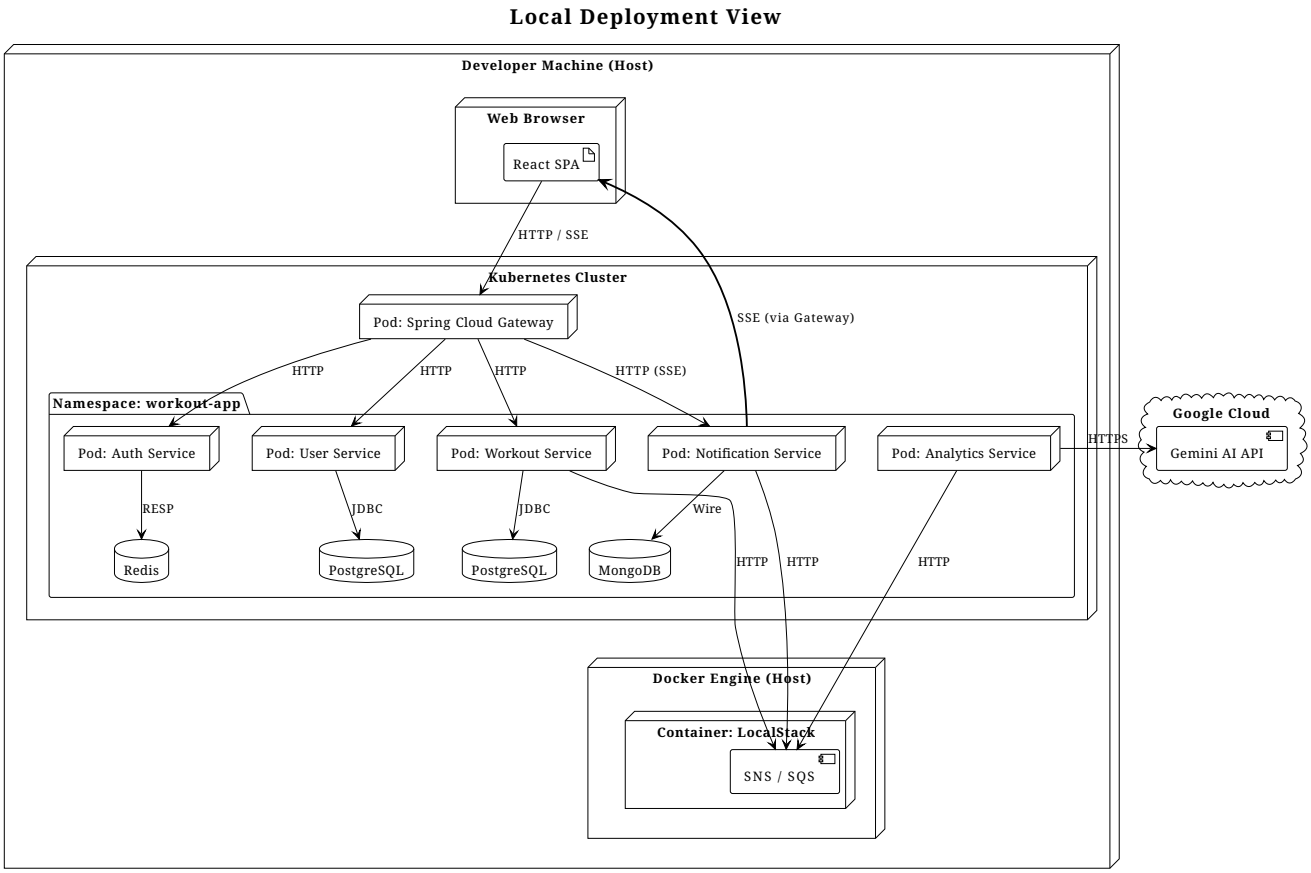
6.2. Create/Update Workout



6.3. Delete Workout



7. Deployment View



7.1. Motivation

In the first iteration, this project will be run locally via Kubernetes and Docker. Kubernetes was chosen to closely resemble a production environment on AWS as well as take advantage of Kubernetes' native service discovery for use with Spring Cloud Gateway. Kubernetes provides the runtime for application services, while LocalStack provides a mock AWS environment. This setup allows for the testing of cloud-native patterns without incurring cloud costs.

Table 21. Infrastructure Elements

Element	Description
Developer Machine (Host)	physical hardware running Docker Desktop or Linux
Kubernetes Cluster	Orchestration layer. Manages workout-app namespace
Spring Cloud Gateway	Ingress point of cluster. Acts as single entry point for web browser
Stateful Persistence	PosgtreSQL, MongoDB, and Redis instances running as containerized workloads. They’re treated as internal cluster resources to simplify networking.
LocalStack	Standalone docker container acting as external cloud provider. Mocks AWS SNS/SQS APIs.

Element	Description
Google Cloud (Gemini API)	External dependency. Requires outbound internet connection and valid API key.

7.2. Quality and/or Performance Features

Scalability

With Kuberentes, each microservice pod can be scaled independently.

Fault Tolerance & Resiliency

The use of SQS as a buffer ensures that messages are not lost if services become unavailable.

Performance

By using asynchronous processing, heavy AI calls are offloaded to a background event driven flow.

Low Latency Notifications

Using SSE to provide real time updates to React SPA without needing constant polling.

8. Cross-cutting Concepts

8.1. Domain Models

The system follows a Database-per-Service pattern to ensure service autonomy and high availability. In this system, the domain model is partitioned across three specialized data stores. To maintain consistency and traceability, the system relies on UUIDs

The `user_id` generated by the User Service acts as the primary ownership key across all services.

The `workout_id` connects a physical workout session to its AI-generated analysis and eventual cleanup.

Table 22. Data Model Mapping

Domain Entity	Storage Type
Primary Role	Lifecycle Policy
Identity & Profile	Postgres (User DB)
Authentication and Authorization	Permanent until account deletion
Workout Session	Postgres (Fitness DB)
Transactional Record	Soft delete (deleted_at); Hard Delete via Cleanup queue
AI Insights	MongoDB (Notification DB)
Read-Optimized Mailbox	TTL Index

8.1.1. Fitness DB (Postgres)

Database for the Workout Management Service

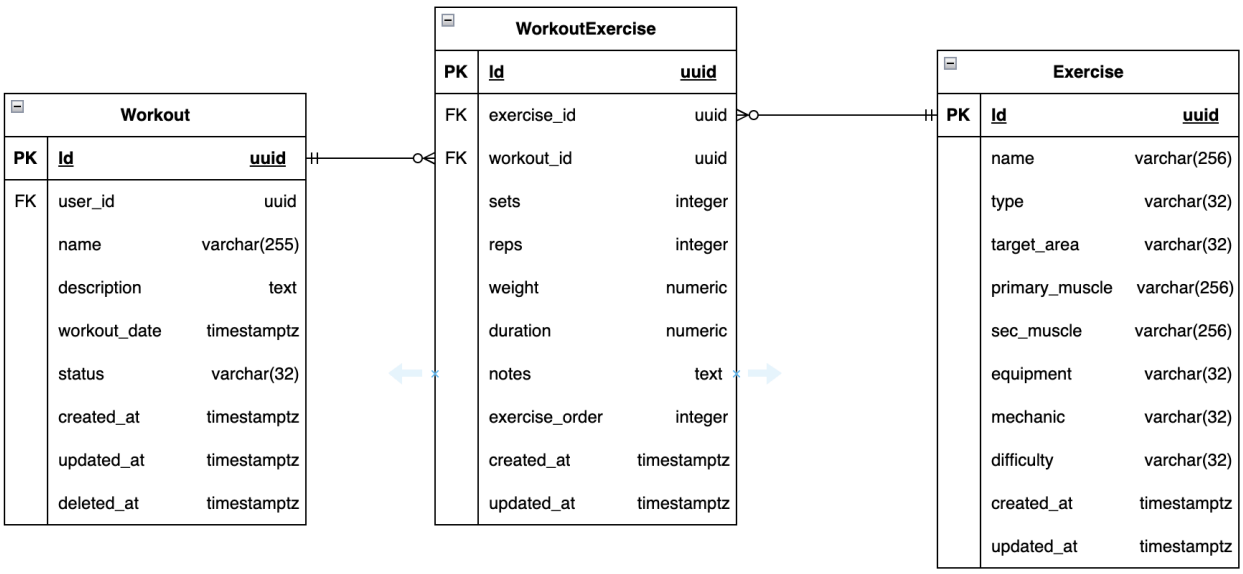


Table 23. Tables

Name	Description
Workout	Represents a single training session for a user. Specifies the name, description, date and status. Also has auditing columns for created and updated.
WorkoutExercise	Links a `Workout` to specific `Exercises`. It tracks performance data for that session, including number of sets, reps and weight used.
Exercises	A global catalog of available exercises to include in a workout. Stores name, target muscle group and difficulty level.

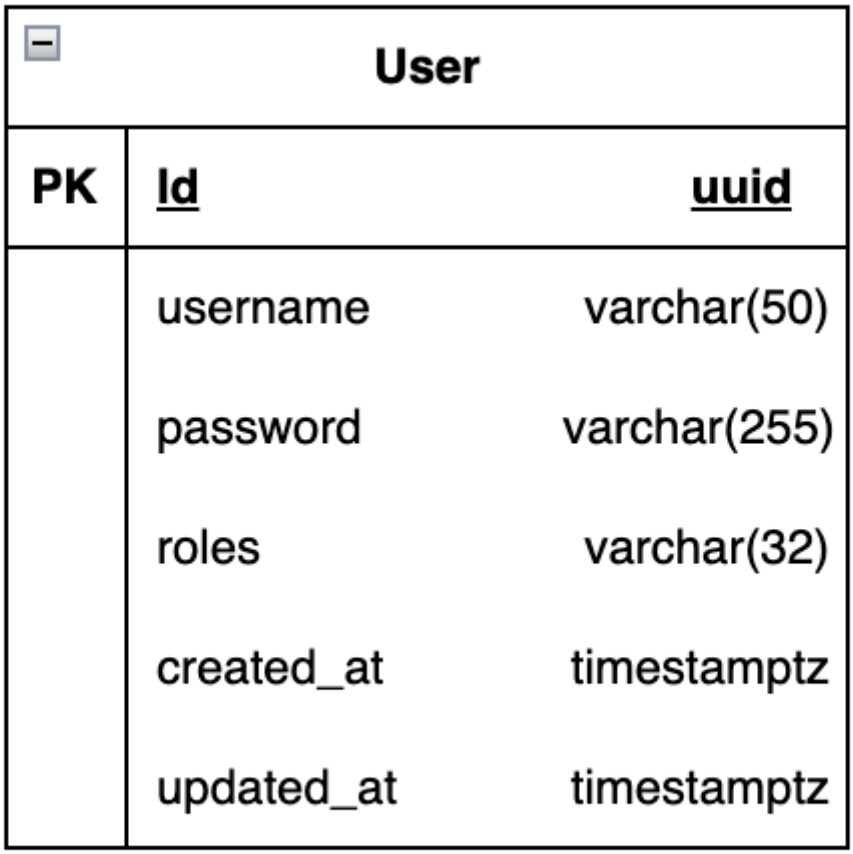
Table 24. Constraints

Table	Constraint	Purpose
Workout	`NOT NULL` on `user_id` and `status`	Ensures every workout is owned by a user and has a lifecycle state
Workout	`CHECK` on `status`	Restricts status column to values of PLANNED, COMPLETED
Exercise	`NOT NULL` on `name`	every exercise must be identifiable
Exercise	`UNIQUE` on `name`	prevents duplicate entries in exercise catalog
WorkoutExercise	`FOREIGN KEY` (`workout_id`) `ON DELETE CASCADE`	Deletes the entry in this table if parent Workout is deleted.
WorkoutExercise	`FOREIGN KEY` (exercise_id) `ON DELETE RESTRICT`	Prevents an exercise from being deleted if its being used in any user workouts.
WorkoutExercise	`CHECK` on `sets`, `reps`, `weight`	Prevent junk data.

TODO: map table to domain model, domain model table, business methods on entity

8.1.2. User DB (Postgres)

Database for the User Service



User DB has one table representing users of the system. Each record represents a unique user, containing their authentication credentials (username and hash of password), user profile and roles.

Table 25. Constraints

Constraint	Purpose
`UNIQUE` for `username`	Prevents multiple accounts from being created with same username.
`NOT NULL` for email and password	Ensures every account has minimum data required for authentication.

8.1.3. Notification DB (Mongodb)

The Notification DB is a stateful buffer between the asynchronous AI processing and the Client. It is a NoSQL document store designed to handle AI-generated insights, which could come in different formats depending on the prompt. The database is optimized for flexible schemas and automated data expiration.

This is the primary collection used by the service:

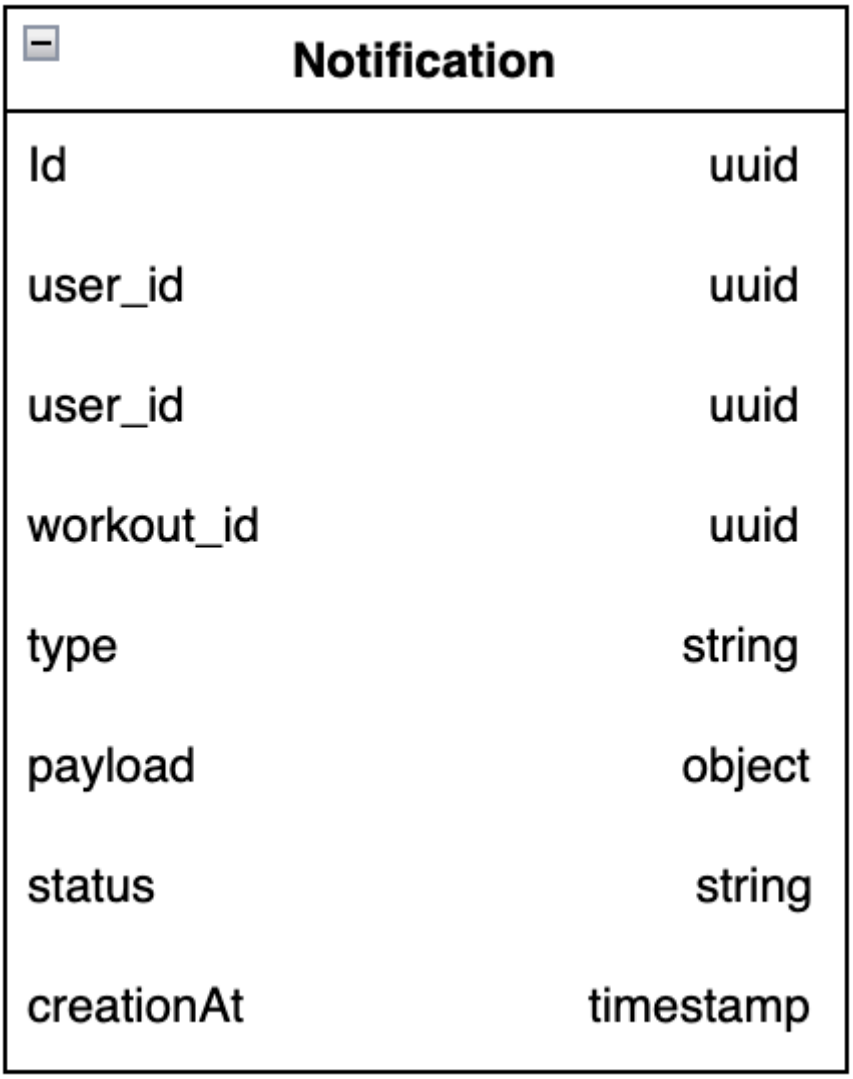


Table 26. Collection

Field	Description
user_id	Correlation Id that links the notification ot its owner.
workout_id	Correlation Id that links the insight to a specific workout session.
type	Category of insight
payload	raw AI output (text, structured JSON, etc)
status	READ or UNREAD
createdAt	point of creation used for TTL

8.1.4. Auth (Redis)

To support the hybrid JWT-Redis security strategy, Redis is used as a high performance session registry.

Implementation

The registry is implemented as a manual key-value store using Spring Data Redis, which ensures a

language-agnostic data format. Time to Live is used and matches the JWT `exp` claim. A logout triggers an immediate deletion of the `jti` key in Redis.

Data Structure:

- **key:** `auth:session:{jti}` where `{jti}` is the unique JWT ID claim
- **value:** a JSON String representing minimal session context: `{"userId": "123", "roles": ["USER"], "issuedAt": …}`

8.2. Persistence

Fitness Tracker uses PostgreSQL for the relational databases, MongoDB for notifications and Redis for sessions.

Access to the databases is through Spring Data JPA.

8.3. Security & Session Handling

This system uses a hybrid JWT-Redis approach.

- **Authentication:** The Authorization Service validates login credentials against the PostgreSQL User DB. It generates a JWT containing a unique `jti` claim and stores the `jti` in Redis using a key value pair where key: `auth:session:{jti}` and value: JSON object representing user session (ie `{"userId": "123", "roles": ["USER"]}`). It returns the JWT back to the client.
- **Session Management:** The API Gateway validates the JWT and then extracts the `jti` and checks that it exists in Redis. If the session is revoked or expired, the request is rejected.
- **Identity Propagation:** To decouple internal services from Auth provider, the Gateway translates the JWT into internal headers. It injects `X-User-ID` and `X-User-Roles` headers into downstream requests. Internal microservices are configured to trust this header from the Gateway identifying the user. Potentially `X-User-Roles` header will be used to pass RBAC roles to be verified by microservices.
- **Authorization (RBAC):** Access control is enforced at the API Gateway based on JWT claims. Microservices may use method-level security with `@PreAuthorize` annotation.

8.4. User Interface

The user interface for Fitness Tracker is a Single Page Application written in React. Both Admin and Users will utilize this UI. The UI will require login for user authentication via the API Gateway.

8.5. Communication and Messaging

The system uses LocalStack to simulate AWS SNS and SQS in a local Docker environment

- **Aysnc Processing:** Long-running AI tasks are offloaded to SQS
- **Correlation IDs:** To ensure traceability across async flow

8.6. Resilience and Fault Tolerance

- **Fault Tolerance (DLQs):** All SQS queues are configured with a Dead Letter Queue. If a service fails to process after 3 retries, the message is moved to the DLQ for manual inspection or later retry. Ordering of messages is not important in this project.
- **Scalability:** All Services are stateless. All session state is externalied to Redis.

8.7. Error Handling and Validity checks

- **Validity Checks:** Handled at Controllers with JSR-303/Bean Validation for immediate Request validation. Also handled at Entity level with database constraints to ensure final data integrity.
- **Global Error Handling:** Each Spring Boot service uses @ControllerAdvice to standardize handling exceptions throughout the system. Allows the Frontend to receive consistent error formats.

8.8. Observability (Loggin and Tracing)

All services log to STDOUT in standardized format. Use a traceId for distributed tracing to link logs across different microservices.

8.9. Configurability and Build Management

- **Java/Spring:** configuration managed via application.yml
- **Python:** Analytics service uses .env files and environment variables
- **Build:** Maven handles Java ecosystem. Docker is common packaging format for all services
- **Testability:** Target of 70% unit test coverage using JUnit5 and Mockito is required for all backend services.

9. Architecture Decisions

9.1. HTTP 202 Accepted

- **Context:** The Gemini API can have high latency (5-15sec). A synchronous call would block the Workout Management Service, leading to poor user experience and potential API Gateway timeouts.
- **Decision:** Use HTTP 202 Accepted. The Workout Management Service acknowledges receipt of request immediately. Processing is offloaded to other services via SNS/SQS.
- **Consequences:** High UI responsiveness; decoupling of services; increased system complexity; requires a solution to notify UI of processing completion

9.2. ADR: SNS + SQS Fanout & Service Decoupling

- **Context:** The Workout Management Service must handle two different workflows, Workout Completion (AI analysis) and Workout Deletion (purge data from notification db). AI processing with Gemini API is high latency.
- **Decision:** Implement a fanout pattern using an SNS topic with message attribute filtering and multiple SQS Queues as dedicated service buffers.
- **Key Design Elements:**
  - **Action-based filtering**: SNS topic broadcasts messages with attributes. Analytics queue filters for messages with `process` attribute to begin AI processing flow. Cleanup queue filters for messages with `delete` attribute to begin cleanup of stored notification messages.
  - **Double Queue Buffer:** A dedicated Notification Queue is placed in between the Analytics Service and the Notification Service. This decouples the AI processing logic from the final notification delivery. It ensures that a surge in AI generated insights doesn't overwhelm the Notification Service's MongoDB instance.
  - **Message Persistence:** SQS ensures messages are not lost if a consumer is temporarily down
- **Rationale:**
  - Extensibility: This decision will make it easy to add new functional streams to the Workout Manangement Service in the future. If a Rewards or Email service was to be added, a new SQS queue could subscribe to the SNS topic, requiring minimal or no changes to the Workout Management Service.
  - Resource Protection: SQS acts as a buffer between high-throughput Workout Management Service and the higher latency downstream resources. By buffering incoming requests, external AI Quotas (Gemini API rate limits) and internal compute resources are protected from traffic spikes.
  - Fault Isolation: The fanout pattern ensures that a failure in one functional stream doesn't impact another.
- Consequences
  - Positive: Enhanced Resiliency and scalability. Clear separation between Java Core, python AI

and MongoDB notification layers.

- Negative: Small delay for AI results, which can be managed on the UI and the 202 Accepted pattern.

9.3. SSE vs websockets

- **Context:** The Client needs to know when the AI analysis is complete to display the results to the User.
- **Decision:** Server-Sent Events (SSE)
- **Consequences:** Unlike Websockets, which are complex and bidirectional, SSE is a lightweight, unidirecitonal push over HTTP. Since the requirement is just a notification from the backend to the client, SSE fits.

9.4. Hybrid JWT-Redis Session Management

- **Context:** Standard JWTs cannot be revoked until they expire, which is a security risk.
- **Decision:** Implement a hybrid opaque-JWT approach described in this guide
- **Consequences:** Combines scalabiilty of JWTs with instant revocation by Redis. JWT's jti is treated as an opaque reference at the API Gateway.

9.5. Gemini API

- **Context:** Gemini Free Tier has strict rate limits and requires high quality, structured output.
- **Decision:** role-based few-shot prompting
- **Strategy:** Describe the context, describe the input format and desired output format with specific desired fields. Provide example of a good response.

10. Quality Requirements

Content

This section contains all relevant quality requirements.

The most important of these requirements have already been described in section 1.2. (quality goals), therefore they should only be referenced here. In this section 10 you should also capture quality requirements with lesser importance, which will not create high risks when they are not fully achieved (but might be *nice-to-have*).

Motivation

Since quality requirements will have a lot of influence on architectural decisions you should know what qualities are really important for your stakeholders, in a specific and measurable way.

Further Information

- See Quality Requirements in the arc42 documentation.
- See the extensive Q42 quality model on https://quality.arc42.org.

10.1. Quality Requirements Overview

Content

An overview or summary of quality requirements.

Motivation

Often we encounter dozens (or even hundreds) of detailed quality requirements. In this overview section you should try to summarize, e.g. by describing categories or topics (as suggested by ISO 25010:2023 or Q42)

If these summary descriptions are already precise, specific enough and measurable, you may skip section 10.2.

Form

Use a simple table in which each line contains a category or topic and a short description of the quality requirement. Alternatively, you may use a mindmap to structure these quality requirements. In literature, the idea of a *quality attribute tree* has also been described, which puts the generic term "quality" as the root and uses a tree-like refinement of the term "quality". [Bass+21] introduced the term "Quality Attribute Utility Tree" for this purpose.

10.2. Quality Scenarios

Content

Quality scenarios make quality requirements concrete and allow to decide whether they are fulfilled (in the sense of acceptance criteria). Ensure that your scenarios are specific and measurable.

Two kinds of scenarios are especially useful:

- *Usage scenarios* (also called application scenarios or use case scenarios) describe the system's runtime reaction to a certain stimulus. This also includes scenarios that describe the system's efficiency or performance. Example: The system reacts to a user's request within one second.
- *Change scenarios* describe the desired effect of a modification or extension of the system or of its immediate environment. Example: Additional functionality is implemented or requirements for a quality attribute change, and the effort or duration of the change is measured.

Form

Typical information for detailed scenarios include the following:

In short form (favoured in the Q42 model):

- **Context/Background**: What kind of system or component, what is the envirionment or situation?
- **Source/Stimulus**: Who or what initiates or triggers a behaviour, reaction or action.
- **Metric/Acceptance Criteria**: A response including a *measure* or *metric*

The long form of scenarios (favoured by the SEI and [Bass+21]) is more detailed and includes the following information:

- **Scenario ID**: A unique identifier for the scenario.
- **Scenario Name**: A short, descriptive name for the scenario.
- **Source**: The entity (user, system, or event) that initiates the scenario.
- **Stimulus**: The triggering event or condition the system must address.
- **Environment**: The operational context or condition under which the system experiences the stimulus.
- **Artifact**: The building-blocks or other elements of the system affected by the stimulus.
- **Response**: The outcome or behavior the system exhibits in reaction to the stimulus.
- **Response Measure**: The criteria or metric by which the system's response is evaluated.

Examples

See the Q42 quality model website for detailed examples of quality requirements.

Further Information

- Len Bass, Paul Clements, Rick Kazman: "Software Architecture in Practice", 4th Edition, Addison-Wesley, 2021.

11. Risks and Technical Debts

This is a new project with no hard deadline, but a list of the tech debt incurred throughout the development process will be kept. Once the basic requirements defined in this document are built, tech debt will be prioritized over new enhancements.

12. Glossary

Contents

The most important domain and technical terms that your stakeholders use when discussing the system.

You can also see the glossary as source for translations if you work in multi-language teams.

Motivation

You should clearly define your terms, so that all stakeholders

- have an identical understanding of these terms
- do not use synonyms and homonyms

Form

A table with columns <Term> and <Definition>.

Potentially more columns in case you need translations.

Further Information

See Glossary in the arc42 documentation.

Term	Definition
<Term-1>	*<definition-1>*
<Term-2>	*<definition-2>*